D4: Implementation Report of a Secure Deep Attestation Framework

DAVINCI: Deep Attestation Framework for Virtualized Network Communication Integrity

University of Padua Technical Leader: Prof. Mauro Conti
Orange Technical Leader: Dr. Ghada Arfaoui
Authors: Rahul Saha, Gulshan Kumar, Marco Pasca
Reviewers: Ghada Arfaoui and Thibaut Jacques, Orange

Version	Date	Authors	Status
v1.0	10/07/2026	Rahul Saha, Gulshan Kumar	Submitted

1 Executive Summary

This deliverable explains how to set up, build, and run the DAVINCI prototype, and how to read the results it produces. It follows the design given in D3 and implements the attestation scheme and the security properties defined in D2. Where D3 describes what each component should do, this document gives the concrete commands that get the framework running on a real machine, together with a worked example that exercises the complete workflow from evidence collection to the final trust decision.

The document has four parts. The first part prepares the environment where DAVINCI runs, either inside a virtual machine created with Vagrant and Ansible, or directly on a macOS or Linux host. The second part sets up DAVINCI itself, meaning the key material and the build of the three services. The third part runs a complete example. The fourth part explains the output of that example and states the current limitations of the prototype.

The prototype consists of three services written in Rust and a small set of Python scripts that act as evidence sources for the demonstration. The Privacy Control Engine on port 3032 applies the privacy transformations to evidence. The Attestation Manager on port 3030 checks each piece of evidence, builds the measurement chain and the trust graph, and exports a signed result. The Verifier on port 3031 checks the signed result and produces the trust decision. All signatures use ECDSA over the NIST P256 curve and all hashing uses SHA-256, which are the cryptographic primitives assumed in D2.

2 System Overview

The concrete DAVINCI instantiation maps the five abstract algorithms of the scheme onto the three services as follows. Setup is the generation of the Attestation Manager key pair, whose public key becomes the trust anchor held by the Verifier. Collect is the per source evidence gathering, where each source signs its measurement with a Root of Trust key and the Attestation Manager verifies that signature before admitting the evidence. Transform is the

privacy processing performed by the Privacy Control Engine. Attest is the construction of the hash linked measurement chain and the signed trust graph. Verify is the Verifier check that combines the signature verification, the freshness check, and the graph validity check into a single decision.

A single attestation run therefore flows through the services in one direction. Evidence enters the Attestation Manager, is authenticated and privacy processed, becomes a node in the measurement chain and a vertex in the trust graph with signed edges, and is finally exported as a signed output that the Verifier appraises against its trust anchor.

3 Setting up the Environment

There are two supported ways to prepare a machine. The first uses a virtual machine and is the most reproducible, because every dependency is installed the same way every time. The second installs the same dependencies directly on macOS or Linux, which is convenient when you already work on the machine day to day.

3.1 A virtual machine with Vagrant and Ansible

This option builds a clean Ubuntu 22.04 virtual machine and provisions it automatically. You need VirtualBox and Vagrant installed on your host. On macOS you can install both with Homebrew.

```
brew install --cask virtualbox
brew install --cask vagrant
```

On a Debian or Ubuntu host you can install them with apt.

```
sudo apt update
sudo apt install -y virtualbox vagrant
```

The repository already contains a `Vagrantfile` and an Ansible playbook at `provisioning/playbook.yml`. From the root of the DAVINCI directory, create and provision the machine with a single command.

```
vagrant up
```

This downloads the Ubuntu image, boots the machine, and then runs the Ansible playbook inside it. The playbook installs the build tools and the OpenSSL development libraries, installs the Rust toolchain, installs Python, generates the DAVINCI keys, and builds the three services. The three service ports 3030, 3031, and 3032 are forwarded to the same ports on your host, so you can reach the services from your own terminal. When provisioning finishes, log in to the machine and move to the project directory, which is mounted inside the machine at `/vagrant`.

```
vagrant ssh
cd /vagrant
```

You are now ready for the example in Section 5. To provision again without rebuilding the machine, run `vagrant provision`. To shut the machine down, run `vagrant halt`, and to delete it completely, run `vagrant destroy`. If you prefer to run the same playbook against a plain Ubuntu host instead of a virtual machine, install Ansible and run it in local mode from the

project directory.

```
sudo apt install -y ansible
ansible-playbook -i "localhost," -c local provisioning/playbook.yml
```

3.2 Setting up directly on macOS or Linux

If you would rather run everything on your own machine, install four things: the Rust toolchain, OpenSSL, pkg-config, and Python 3. Install the Rust toolchain through rustup, which does not require administrator rights.

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
source "$HOME/.cargo/env"
```

On macOS, install OpenSSL and pkg-config with Homebrew, then point pkg-config at the OpenSSL you just installed so that the Rust build can find it.

```
brew install openssl@3 pkg-config
export PKG_CONFIG_PATH="$(brew --prefix openssl@3)/lib/pkgconfig"
```

On Ubuntu or Debian, install the equivalent packages with apt.

```
sudo apt update
sudo apt install -y build-essential pkg-config libssl-dev openssl python3
```

Python 3 is already present on macOS and on most Linux systems. The demonstration scripts use only the Python standard library and the openssl command line tool, so there are no extra Python packages to install. You can confirm the toolchain is ready with the following commands.

```
rustc --version
openssl version
python3 --version
```

4 Setting up DAVINCI

With the environment in place, move to the root of the DAVINCI directory. Inside the virtual machine this is `/vagrant`. The `attestation-manager`, `verifier`, and `pce-service` folders each hold one Rust service. The `keys` folder holds the key material. The `demo` folder holds the example scripts. The `docs` folder holds the design and implementation reports.

4.1 Generating the keys

DAVINCI relies on two kinds of keys. The Attestation Manager has its own key pair. It signs its results with the private key, and the Verifier trusts the matching public key, which plays the role of the trust anchor in D2 and D3. There is also one key pair for each Root of Trust source, such as the TPM and the confidential computing report. In a real deployment these would be hardware protected keys inside the attestation hardware. In the prototype they are software keys that stand in for that hardware, so that the signature checks are genuine even without special hardware. Generate all of the keys with the helper script.

```
./keys/gen-keys.sh
```

This writes the Attestation Manager key pair and the per source key pairs into the keys folder. The private keys are never committed to version control, so this step must be run once on any fresh copy of the repository.

4.2 Building the services

The simplest path is the Makefile target that generates the keys and then builds all three services.

```
make build
```

On macOS, remember to export `PKG_CONFIG_PATH` first, as shown in Section 3, so that the OpenSSL library is found during the build. The first build downloads and compiles the dependencies and can take a few minutes. Later builds are fast. If you prefer to build the services one at a time, you can build each crate on its own.

```
cd pce-service && cargo build && cd ..
cd attestation-manager && cargo build && cd ..
cd verifier && cargo build && cd ..
```

4.3 Configuration

The services read a few settings from environment variables. The defaults are chosen so that everything works on one machine without any configuration, so you can usually ignore this section for a first run.

Variable	Default	Purpose
`DAVINCI_KEYS_DIR`	`../keys`	Folder that holds the key material.
`PCE_URL`	`http://localhost:3032`	Where the Attestation Manager reaches the Privacy Control Engine.
`ATTESTATION_MANAGER_URL`	`http://localhost:3030`	Where the Verifier reaches the Attestation Manager.
`DATABASE_URL`	unset	PostgreSQL connection for the Privacy Control Engine. If unset, it keeps data in memory.
`AM_BIND`	`0.0.0.0:3030`	Listen address of the Attestation Manager.
`VERIFIER_BIND`	`0.0.0.0:3031`	Listen address of the Verifier.
`PCE_BIND`	`0.0.0.0:3032`	Listen address of the Privacy Control Engine.

5 Running an Example Instance

The example drives the whole workflow. It sends platform, hypervisor, guest, and runtime evidence to the Attestation Manager, asks the Verifier for a decision, and prints the result. It runs two scenarios one after the other. The first uses authentic evidence and should be trusted. The second tampers with the guest evidence after the Root of Trust has signed it, and should be rejected.

5.1 Starting the services

Start the three services in the correct order. Start the Privacy Control Engine first, because the Attestation Manager depends on it. Give each service its own terminal, because each one runs in the foreground and prints a log.

```
make run-pce                  # terminal 1, port 3032
make run-attestation-manager  # terminal 2, port 3030
make run-verifier             # terminal 3, port 3031
```

You can confirm that all three are up with three quick requests. The first returns a status of ok, the second returns the Attestation Manager public key in PEM form, and the third returns a status of ok.

```
curl -s http://127.0.0.1:3032/health
curl -s http://127.0.0.1:3030/pubkey
curl -s http://127.0.0.1:3031/health
```

5.2 Running the example

In a fourth terminal, move to the demo folder and run the comparison example, which runs the trusted scenario and then the tampered scenario.

```
cd demo
python3 demo_comparison.py
```

There is also a single end to end run that only performs the trusted scenario, available as `python3 demo_scenario.py`. The same runs are available through the Makefile as `make demo-compare` and `make demo`.

6 Reading the Output and Current Limitations

6.1 The trusted scenario

In the trusted scenario every layer sends authentic evidence, so the Attestation Manager admits each item, builds the graph, and signs it. The Verifier then confirms every check and returns a trusted decision.

```
Trust Decision: TRUSTED
Checks:
  PASS Output Signature (ECDSA-P256)
  PASS Freshness (challenge binding)
  PASS Platform Root Present
  PASS Edge Signatures (ECDSA-P256)
  PASS Layer Monotonicity
  PASS Binding Policy
  PASS Acyclicity
  PASS Cross-Layer Trust Continuity
```

6.2 The tampered scenario

In the tampered scenario the guest measurement is changed after the Root of Trust signed it. The Attestation Manager checks the signature over the guest measurement, finds that it no longer matches, and rejects that evidence. The guest node therefore never enters the graph. The runtime node above it then has no path down to the platform, so the Verifier reports that the cross layer trust continuity check has failed and returns a not trusted decision.

```
3. Guest SEV-SNP evidence ...  (TAMPERED after RoT signature)
   AM: rejected_auth: RoT signature invalid, evidence not authentic (Auth = 0)

Trust Decision: NOT TRUSTED
  PASS Output Signature (ECDSA-P256)
  PASS Freshness (challenge binding)
  PASS Platform Root Present
  PASS Edge Signatures (ECDSA-P256)
  PASS Layer Monotonicity
  PASS Binding Policy
  PASS Acyclicity
  FAIL Cross-Layer Trust Continuity  (Application node has no path to Platform)
```

The important point is that this decision comes from real signature checks and graph checks, and not from any label attached to the session.

6.3 The signed output and the trust graph

The Attestation Manager exports the result at the endpoint `GET /attest/output/{session}`, which takes the challenge as a `nonce` query parameter. The response contains the output and a proof, which is the Attestation Manager signature over that output. A trimmed example follows.

```
{
  "out": {
    "sid": "d4_example_1783669785",
    "challenge": "chal_demo_0001",
    "ts_out": 1783669785,
    "manifest": { "redacted": 8, "aggregated": 0, "retained": 1 }
  },
  "pf": "304602210082649319d58a1a7ae4d97faa074fe3cefd343c..."
}
```

The manifest records what the Privacy Control Engine did to the evidence. In this run it removed eight sensitive fields across the four evidence items, which are the host name and the address field of each item, and kept the fields that verification needs. Each node in the graph carries its profile, its measurement, its position in the hash chain, and its hash.

```
{
  "jti": "645f3ed0-e43c-4334-a3e0-4f8c0182dd5d",
  "eat_profile": "layer:Application,mode:Runtime,source:Ima",
  "eat_meas": "{ \"claims\": {}, \"measurement\": \"bf1f8f7c3bba...\" }",
  "iat": 1783669785,
  "x_davinci_prev": "
    f792985febed579dd70c0e110474e59268dedc84717a19255fa9ea9496e93859",
  "hash": "f9a9e69f02e52dcbf3ba7c54b5c341983aa0c3fadbe23e64ae2b3bebf5e980ef"
}
```

Each edge records a trust dependency and carries the Attestation Manager signature over that edge.

```
{
  "from": "4165ca62-aa7b-46c8-ae29-5fe7566a7e65",
  "to": "5bec0611-88bf-4992-8ca7-a3e492f3c2c2",
  "relationship": "VmIncludesHypervisor",
  "signature": "30440220429844689d9cf0dfda5a918ebb3592e794a1c929..."
}
```

6.4 How the checks map to the security properties

Each check the Verifier performs corresponds to a property defined in D2.

Verifier check	D2 property
Output Signature, Edge Signatures	Authenticity, since only the Attestation Manager private key can produce them.
Freshness	Freshness and replay resistance, through the fresh challenge bound into the signed output.
Platform Root, Layer Monotonicity, Acyclicity, Binding Policy, Cross-Layer Trust Continuity	Deep integrity and cross layer trust continuity, through a signed and well formed path from every layer to the platform anchor.
Rejection of tampered evidence at the Attestation Manager	Evidence authenticity that underlies multi modal soundness.
Privacy Control Engine transform	The privacy properties, through pseudonymisation and field suppression.

The binding policy check corresponds to the admissible edge rule Π_{bind} of D3, which permits only the expected parent relationships between layers.

6.5 The application interfaces

The Attestation Manager accepts boot evidence at `POST /attest/boot` and runtime evidence at `POST /attest/runtime`. It returns the signed output at `GET /attest/output/{session}`, the raw graph for inspection at `GET /arg/{session}`, and its public key at `GET /pubkey`. The Verifier accepts a request at `POST /verify` with a session and a challenge and returns the full decision with the per check details. The Privacy Control Engine accepts `POST /privacy/transform`, `POST /validate/node`, and `POST /resolve/pseudonym`, and reports its state at `GET /health`.

6.6 Current limitations

The prototype is faithful to the design in the parts it implements. The following points describe what it does not yet do, so that results are read correctly.

First, the hardware Roots of Trust are simulated. The evidence is signed with software ECDSA keys that stand in for hardware protected keys, so the signature checks are real, but no physical TPM or confidential computing hardware is involved.

Second, the bootstrap attestation of the Attestation Manager host, described in D3, is not implemented. The Verifier trusts the Attestation Manager public key from its key folder rather than first appraising the host that holds it.

Third, the Verifier validates the graph it receives, but it does not yet require a fixed set of layers to be present. A result that contains a smaller but internally valid graph is accepted as far as it goes. Enforcing a required set of layers per session is the natural next step and follows directly from the session policy described in D3.

6.7 Stopping the services

Each service runs in the foreground, so stopping it is a matter of pressing Ctrl and C in its terminal. If you started the services in the background, you can stop them by name.

```
pkill -f target/debug/pce-service
pkill -f target/debug/attestation-manager
pkill -f target/debug/verifier
```

7 Conclusion

This report has shown how to prepare an environment for DAVINCI, how to build the three services and their key material, and how to run a complete example that produces a cryptographically justified trust decision. The worked example demonstrates that authentic evidence is accepted and that tampered evidence is detected and rejected, in both cases through real signature and graph checks rather than any external label. Together with the design in D3 and the theory in D2, this completes the description of the DAVINCI prototype and provides a reproducible basis for further development.